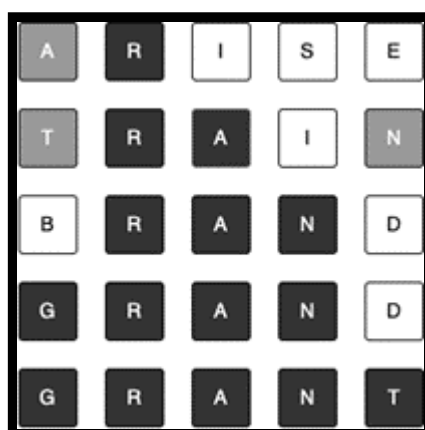


Université Sorbonne Paris Nord – IUT de Villetaneuse
Ressource : R201 – Base de la programmation objet
Situation d'Apprentissage et d'Évaluation
Auteur du sujet : B. Lemaire & H. AZZAG
Fraude : Tout plagiat entraînera l'application du règlement des études. Toute suspicion de fraude pourra donner lieu à un signalement et à une procédure disciplinaire.

Le jeu Find My Word



1. Objectifs d'une SAÉ (Situation d'Apprentissage et d'Évaluation)

- **Mobiliser des ressources** (connaissances, savoir-faire, savoir-être) dans une situation concrète.
- **Développer des compétences professionnelles** en lien avec le référentiel du BUT Informatique 1ère année.
- **Produire une réalisation tangible** (application, programme, rapport, prototype...)
- **Apprendre à travailler en autonomie et/ou en équipe.**
- **Respecter un cahier des charges** (contraintes techniques, délais, qualité)
- **Adopter une démarche de projet** (analyse, conception, réalisation, validation)
- **Communiquer efficacement** à l'écrit et/ou à l'oral.
- **Être évalué en situation réelle** sur la base de critères explicites (compétences, qualité, démarche)

2. Déroulement d'une SAÉ

2.1. Prise de connaissance du sujet

- Lecture attentive du **cahier des charges**.
- Identification des **attendus**, des **contraintes** et des **livrables**.
- Clarification des zones d'ombre (questions à poser si nécessaire).

2.2. Analyse et organisation

- Découpage du travail en **tâches concrètes**.
- Identification des **compétences à mobiliser**.

- Choix des **outils** et des **technologies**.

L'autonomie est primordiale : vous devez être capable de comprendre le sujet et de structurer votre travail sans attendre des consignes détaillées à chaque étape.

2.3. Planification

- Élaboration d'un **planning réaliste**.
- Répartition des tâches entre les membres de l'équipe de SAE.
- Définition de **jalons intermédiaires**.

La planification est essentielle : une mauvaise gestion du temps entraîne presque toujours un travail incomplet ou de faible qualité.

2.4. Réalisation

- Développement progressif de la solution.
- Tests réguliers et ajustements.
- Respect des **bonnes pratiques** (qualité du code, lisibilité, organisation)

Le projet devra être conçu de manière structurée, en respectant les principes de la **programmation orientée objet**, notamment :

- l'encapsulation ;
- l'héritage ;
- le polymorphisme ;
- la séparation des responsabilités.

2.5. Vérification et amélioration

- Validation du fonctionnement global.
- Relecture du travail (code, rapport, livrables)
- Corrections et optimisations.

2.6. Livraison

- Préparation des livrables attendus (code, documentation, rapport...)
- Vérification de la conformité avec le cahier des charges.
- Respect des **délais imposés**.

2.7. Présentation / Évaluation

- Présentation du travail réalisé (orale et/ou écrite)
- Justification des choix techniques et organisationnels.
- Retour critique sur le travail effectué.

Elements clefs :

- **Autonomie** : vous êtes acteur de votre apprentissage.
- **Planification** : anticiper = réussir

- **Rigueur** : qualité du travail et respect des consignes.
- **Engagement** : implication individuelle et/ou collective indispensable.

3. Le jeu FindMyWord

3.1 Présentation du jeu FindMyWord

L'application est basée sur un jeu populaire de devinette de mots *Wordle*. Les joueurs essaient de deviner un mot de cinq lettres. Des points sont attribués en fonction de la rapidité avec laquelle un joueur devine le mot. Le joueur reçoit des retours sur chaque tentative pour l'orienter vers la bonne réponse.

Description des règles de FindMyWord

Pour jouer à **FindMyWord**, un joueur dispose de six tentatives pour deviner un mot de cinq lettres. Après chaque tentative, les lettres du mot sont mises en évidence comme suit :

- La lettre correcte à la bonne position est étiquetée (**OK**)
- La lettre correcte à la mauvaise position est étiquetée (**PRESENT**)
- Les lettres incorrectes qui ne figurent pas dans le mot secret est étiquetée (**ABSENT**)

Le joueur peut utiliser ces indications pour faire une meilleure supposition lors de sa prochaine tentative. Une fois qu'un joueur devine correctement le mot, il marque des points. Il obtient six points pour une bonne réponse au premier essai, cinq points pour une bonne réponse au deuxième essai, et un point pour une bonne réponse au sixième et dernier essai. Les joueurs s'affrontent lors de différentes manches pour obtenir le score le plus élevé.

Notes :

- Vous allez développer une version simplifiée de ce jeu.
- Il n'est pas nécessaire d'avoir déjà joué à *Wordle* ou à un jeu de devinette de mots pour réaliser ce projet.
Toutes les règles nécessaires sont décrites dans le sujet (voir en annexes pour les règles du jeu, les cas de test et les exemples de déroulement).

Le joueur devra proposer des mots et interpréter les retours du programme (**OK / PRESENT / ABSENT**) afin de progresser dans la partie.

3.2 Quelles sont les principales étapes dans un tour de jeu ?

Le programme joue le rôle d'arbitre :

- il choisit un mot secret;
- il analyse les tentatives du joueur au maximum 6 tentatives ;
- il indique pour chaque lettre si elle est correcte, mal placée ou absente ;
- il gère le nombre d'essais ;
- il détecte la fin de partie (victoire ou défaite).

Optionnellement (cela sera apprécié par le jury)

- il gère le temps pour faire un tour de jeu ;
- il gère les tours entre deux joueurs et cumule les points et les temps pour chaque joueur ; le nombre de tours est décidé parmi 1, 5, 10, 20 ;
- maintient un historique des parties.

3.2.1 Voici les fonctionnalités principales du programme

- **demander** et **enregistrer** le nom du joueur,
- **afficher** (en mode texte) l'état courant de la partie sous forme de grille,
- afficher (optionnellement) des informations complémentaires sur la partie, par exemple le nombre d'essais déjà utilisés, le nombre d'essais restants ou le temps total de jeu,
- **afficher les tentatives déjà jouées** par le joueur,
- **saisir la tentative** proposée par le joueur,
- **valider ou invalider** la tentative proposée,
- analyser le mot proposé et afficher, pour chaque lettre, le résultat de l'analyse (**OK / PRESENT / ABSENT**),
- mettre à jour l'affichage de la grille après chaque tentative valide,
- **détecter les situations de fin de partie** :
 - victoire si le mot secret est trouvé,
 - défaite si le nombre maximal d'essais est atteint,
 - afficher un message de fin de partie indiquant clairement le résultat obtenu.

La section suivante décrit le déroulement concret d'une partie.

3.2.2 Déroulement concret d'une partie

En début de partie, le programme choisit aléatoirement un mot secret de 5 lettres parmi les mots qui se trouve dans un fichier au format **JSON** fourni.

Note : Vous disposerez de la *javadoc* et du *bytecode* de la classe **RetrieveWordsFromJSON** vous permettant d'interagir avec un fichier **JSON** de mots.

La grille de jeu est initialisée (6 lignes × 5 colonnes vides).

Dès que la partie commence, le programme :

1. affiche l'état courant de la partie (grille et tentatives déjà effectuées),
 2. saisit la tentative proposée par le joueur,
 3. vérifie la validité de la tentative et, tant qu'il n'est pas valide, redemande une saisie,
 4. analyse le mot proposé et affiche le résultat pour chaque lettre (**OK / PRESENT / ABSENT**),
 5. enregistre la tentative dans la grille,
 6. vérifie si la partie est terminée.
-

4. Organisation de votre travail

La SAE se déroule selon **trois axes menés en parallèle**, qui doivent avancer de manière coordonnée tout au long du projet :

4.1 Trois grandes tâches

4.1.1 La modélisation (en amont et en continu)

- La **modélisation doit impérativement être réalisée AVANT l'implémentation**
- Elle consiste à définir l'architecture logicielle de l'application à développer (UML, classes, relations, responsabilités).
- Elle sert de **guide de conception** et permet d'éviter les erreurs dès le départ.
- Elle peut évoluer, mais ne doit jamais être réalisée a posteriori pour "justifier" le code.

4.1.2 L'implémentation

- Développement progressif **à partir de la modélisation**.
- Respect des choix de conception définis en amont.
- Amélioration continue du code (lisibilité, structure, qualité).

4.1.3 La rédaction du rapport

- Le rapport doit être **rédigé tout au long du projet**, et non à la fin.
- Il permet de :
 - expliquer les choix réalisés ;
 - justifier les décisions techniques ;
 - présenter les difficultés rencontrées et les solutions apportées ;

Une rédaction tardive conduit généralement à un travail superficiel et incohérent.

Elements clefs :

Ces trois axes ne sont pas indépendants :

- la **modélisation guide l'implémentation** ;
- l'**implémentation alimente le rapport** ;
- le **rapport formalise votre démarche** .

Une SAE réussie repose sur l'équilibre et l'avancement simultané de ces trois dimensions.

4.2 Décrivons en détail chaque tâche :

4.2.1 Modélisation

Il est important de correctement définir les objets du problème impliqués dans une partie de FindMyWord (joueur, mot, partie, source de mots, etc.).

Chaque type d'objet sera représenté par une classe dans la modélisation.

Ainsi vous devez au minimum considérer :

— une classe **Game** dont la responsabilité est de gérer la partie, les essais, la validation des propositions, l'analyse des mots saisis, l'affichage de la progression et la détection de la fin de partie,

- une classe **Word** représentant un mot du jeu. Cette classe encapsule les règles métier et permettre de vérifier entre autre la validité d'un mot.
- une classe **WordRepository** permettant de récupérer un mot parmi une liste de mots :

Cas possibles :

- récupération aléatoire (jeu réel)
 - récupération d'un mot fixe (tests)
- une classe **Main** permettant de lancer le programme.

Pour modéliser l'architecture logicielle de l'application, vous devrez utiliser la méthode suivante :

1. Proposer un premier **diagramme de classes UML simplifié** faisant apparaître uniquement les liens entre classes (associations et héritages) sans préciser les attributs (variables) ni les opérations (méthodes).
2. Après analyse du fonctionnement du jeu, ajouter les attributs et les opérations.

Consignes :

- Ne pas modifier le diagramme de classes simplifié.
- Faire apparaître chaque classe sur un autre diagramme avec ses attributs et opérations (dont les constructeurs).

4.2.2 Implémentation

Quelques bonnes pratiques pour l'implémentation de l'application

L'implémentation Java s'effectue **APRÈS la modélisation UML** et doit respecter cette dernière.

Si le modèle UML s'avère insuffisant ou incorrect, vous devez le **corriger AVANT de poursuivre l'implémentation**.

Toute rétro-analyse consistant à concevoir le modèle UML **à partir du code Java** est très visible et sera **FORTEMENT PÉNALISÉE**.

L'utilisation du **polymorphisme est obligatoire** dès que cela est pertinent.

Un code propre et bien construit est primordial ; ces aspects seront évalués :

- Le code doit être **lisible et structuré** :
 - création de packages explicites.
- les méthodes doivent être **courtes et simples** (une seule responsabilité)
- le **code doit être auto-descriptif** (il doit raconter l'histoire de ce qu'il fait)
 - les classes, variables et méthodes doivent avoir des **noms explicites** c'est à dire indiquer leur intention.

- **les commentaires doivent rester exceptionnels** pour des parties du code où sa seule lecture ne suffit pas.

L'affichage du jeu (grille, messages, interactions utilisateur) doit être, autant que possible, **séparé de la logique métier**.

Autrement dit :

- la classe **Game** gère la logique du jeu,
- l'affichage doit rester simple et ne pas mélanger calcul et présentation.

Attention ! Un code de mauvaise qualité sera fortement sanctionné même s'il fonctionne.

Données

Le projet utilisera un fichier **words.json** fourni contenant une liste de mots utilisables par le programme.

Exemple :

```
["train", "sable", "ligne", "route", "pince"]
```

Le mot est choisi aléatoirement dans la liste.

5. Une SAE demande une organisation rigoureuse du travail

Le développement de l'application s'effectue en **binôme**.

Afin d'optimiser l'efficacité du travail et d'éviter les redondances, il est attendu que chaque membre du binôme prenne en charge **une partie clairement identifiée de l'application**, en cohérence avec le découpage en classes.

Une répartition claire des responsabilités est essentielle :

- elle permet d'éviter le **double travail** ;
- elle facilite la **coordination** ;
- elle garantit une **meilleure qualité globale**.

Cette organisation n'exclut pas les échanges : les choix doivent être **discutés et validés ensemble** afin d'assurer la cohérence de l'ensemble du projet.

6. Modalités de remise du projet

Le dossier de rendu comprendra :

1. Le code Java de toutes les classes sous forme d'un fichier ZIP (**obligatoire**)

2. Un document PDF (**obligatoire**) comprenant :

- vos **noms, prénoms et groupe**,
- une indication précisant **comment lancer le programme**.

NOTE IMPORTANTE : LE PROGRAMME DOIT ÊTRE LANCÉ DEPUIS UN TERMINAL sans avoir besoin d'ouvrir un environnement de développement intégré (IDE) type Eclipse ou IntelliJ.

- les **diagrammes de classes UML**,
- un **bilan** précisant :

- ce qui a été fait,
- ce qui n'a pas été fait,
- ce qui fonctionne,
- ce qui ne fonctionne pas,
 - une **description de votre organisation de travail** (répartition du travail, difficultés rencontrées, choix réalisés, ...)

Ces deux documents (ZIP et PDF) doivent être envoyés par courriel à votre chargé de travaux dirigés au plus tard le **12 juin 2026 minuit**.

Aucun envoi hors délai ne sera accepté.

7. Évaluation du projet

Seront évalués :

- la qualité de la **modélisation UML**,
- la qualité de l' **implémentation** (lisibilité, structure, respect des principes POO),
- le bon fonctionnement du programme,
- le respect des contraintes du sujet,
- la qualité du bilan fourni.

Les programmes seront vérifiés par des outils de **détection de plagiat**. Tout plagiat (entre étudiants ou via une intelligence artificielle) sera **très fortement pénalisé**.

NOTE IMPORTANTE

La note est **individuelle** :

- Vous devez être capable d'expliquer votre code, de justifier vos choix de conception et de réaliser des modifications lors de la soutenance.
- Un étudiant incapable d'expliquer son code ou de le modifier pourra être pénalisé, même si le programme fonctionne.

ANNEXE A — Règles du jeu FindMyWord

Le jeu FindMyWord se joue entre un joueur et le programme.

Le programme choisit un mot secret de 5 lettres que le joueur doit deviner.

On considère qu'une lettre apparaît au maximum une seule fois dans le mot, ce qui simplifie l'analyse.

À chaque tour, le joueur propose un mot.

Le programme analyse cette proposition et indique, pour chaque lettre :

- si elle est correcte et bien placée (**OK**),
- si elle est présente dans le mot mais mal placée (**PRESENT**),
- si elle est absente du mot (**ABSENT**).

L'objectif du joueur est de retrouver le mot secret en un nombre limité d'essais.

A.1 La grille de jeu

La partie se déroule sur une grille composée de :

- **6 lignes** correspondant aux tentatives du joueur,
- **5 colonnes** correspondant aux lettres du mot.

Chaque ligne représente un essai complet du joueur.

A.2 Les règles du mot

Le mot secret respecte les contraintes suivantes :

- il contient **exactement 5 lettres**,
- il ne contient **aucune lettre répétée**,
- il est issu d'une liste de mots fournie (**words.json**).

La tentative proposée par le joueur doit également respecter ces contraintes.

A.3 Déroulement d'un tour

À chaque tour :

1. le joueur saisit une tentative,
 2. le programme vérifie la validité de la tentative,
 3. si la tentative est invalide, une erreur est affichée et le joueur doit recommencer,
 4. si la tentative est valide : — le programme analyse chaque lettre,
 - le résultat est affiché,
 - la grille est mise à jour.
-

A.4 Analyse des lettres

Pour chaque lettre de la tentative proposée :

- **OK** : la lettre est à la bonne position dans le mot secret,
 - **PRESENT** : la lettre existe dans le mot mais à une autre position,
 - **ABSENT** : la lettre n'existe pas dans le mot secret.
-

A.5 Validité d'une proposition

Une proposition est valide si :

- elle contient exactement **5 lettres**,
- elle ne contient que des caractères alphabétiques,
- elle ne contient **aucune lettre répétée**.

Une proposition invalide ne doit **pas être comptabilisée comme un essai**.

A.6 Fin de partie

La partie se termine dans les cas suivants :

- **Victoire** : le joueur trouve le mot secret,
- **Défaite** : le joueur utilise ses **6 tentatives** sans trouver le mot.

Dans tous les cas, le programme doit afficher un message de fin de partie.

A.7 Remarques

- L'affichage doit rester **clair et compréhensible**,
- La grille doit permettre de visualiser les tentatives précédentes,
- Les informations essentielles doivent toujours être visibles (essais, résultats, progression).

ANNEXE B — Cas de test officiels

Les cas de test suivants permettent de vérifier le bon fonctionnement du programme.

B.1 Validation des entrées

Cas 1 — mot trop court

Entrée :

chat

Sortie attendue :

Erreur : le mot doit contenir exactement 5 lettres.

Cas 2 — mot trop long

Entrée :

avions

Sortie attendue :

Erreur : le mot doit contenir exactement 5 lettres.

Cas 3 — caractères non alphabétiques

Entrée :

ab1de

Sortie attendue :

Erreur : le mot doit contenir uniquement des lettres.

Cas 4 — lettres répétées

Entrée :

belle

Sortie attendue :

Erreur : le mot ne doit pas contenir de lettre répétée.

B.2 Analyse des propositions

Mot secret : ligne

Cas 5 — toutes les lettres correctes

Entrée :

ligne

Sortie attendue :

[L][I][G][N][E] → OK OK OK OK OK

Cas 6 — lettres partiellement correctes

Entrée : table

Sortie attendue :

[T][A][B][L][E] → ABSENT PRESENT ABSENT PRESENT ABSENT

Cas 7 — mélange OK / PRESENT / ABSENT

Entrée :

lampe

Sortie attendue :

[L][A][M][P][E] → PRESENT ABSENT ABSENT PRESENT

Cas 8 — lettres présentes mais mal placées

Entrée :

genil

Sortie attendue :

[G][E][N][I][L] → PRESENT PRESENT PRESENT PRESENT

Cas 9 — mot trouvé au dernier essai avec affichage complet de la grille

Entrées successives :

pomme
table
lampe
genil
pluie
ligne

Sortie attendue :

[P] [O] [M] [M] [E]	→ ABSENT	ABSENT	ABSENT	ABSENT	PRESENT
[T] [A] [B] [L] [E]	→ ABSENT	ABSENT	ABSENT	PRESENT	ABSENT
[L] [A] [M] [P] [E]	→ OK	ABSENT	ABSENT	ABSENT	ABSENT
[G] [E] [N] [I] [L]	→ PRESENT	PRESENT	PRESENT	PRESENT	PRESENT
[P] [L] [U] [I] [E]	→ ABSENT	PRESENT	ABSENT	PRESENT	ABSENT
[L] [I] [G] [N] [E]	→ OK	OK	OK	OK	OK

B.3 Gestion des essais

Cas 10 — saisie invalide non comptabilisée

Entrées : chat
ligne

Comportement attendu :

- "chat" → erreur
- "ligne" → essai 1

Cas 11 — victoire

Entrées : table
lampe
ligne

Sortie attendue : Bravo !
Vous avez trouvé le mot en 3 essais.

Cas 12 — défaite

Entrées : table
lampe

lever
loger
luire
livre

Sortie attendue : Vous avez utilisé tous vos essais.
Partie perdue.
Le mot secret était : ligne

B.5 Remarques

- Les erreurs ne doivent pas consommer un essai
- L’analyse doit être correcte
- L’affichage doit être lisible

ANNEXE C — Exemples de parties (traces console)

C.1 Exemple — Partie gagnée

```
=====
FIND MY WORD - BUT1
=====

Entrez votre nom : Alice

Bonjour Alice

[ ][ ][ ][ ][ ] → *****  *****  *****  *****  *****

Tentative 1 : table

[ T ][ A ][ B ][ L ][ E ] → ABSENT  ABSENT  ABSENT  PRESENT  ABSENT
-----

Tentative 2 : lampe

[ T ][ A ][ B ][ L ][ E ] → ABSENT  ABSENT  ABSENT  PRESENT  ABSENT
[ L ][ A ][ M ][ P ][ E ] → OK      ABSENT  ABSENT  ABSENT  ABSENT
-----

Tentative 2 : ligne

[ T ][ A ][ B ][ L ][ E ] → ABSENT  ABSENT  ABSENT  PRESENT  ABSENT
[ L ][ A ][ M ][ P ][ E ] → OK      ABSENT  ABSENT  ABSENT  ABSENT
[ L ][ I ][ G ][ N ][ E ] → OK      OK      OK      OK      OK
```

Bravo Alice !
Vous avez trouvé le mot en 3 essais.
Vous avez acquis : 4 points

C.2 Exemple — Partie perdue

```
=====
FIND MY WORD – BUT1
=====

Entrez votre nom : Alice

Bonjour Alice

[ ][ ][ ][ ][ ] → *****  *****  *****  *****  *****

Tentative 1 : table

[ T ][ A ][ B ][ L ][ E ] → ABSENT  ABSENT  ABSENT  PRESENT  ABSENT
-----

Tentative 2 : lampe

[ T ][ A ][ B ][ L ][ E ] → ABSENT  ABSENT  ABSENT  PRESENT  ABSENT
[ L ][ A ][ M ][ P ][ E ] → OK      ABSENT  ABSENT  ABSENT  ABSENT
-----

Tentative 2 : ligne

[ T ][ A ][ B ][ L ][ E ] → ABSENT  ABSENT  ABSENT  PRESENT  ABSENT
[ L ][ A ][ M ][ P ][ E ] → OK      ABSENT  ABSENT  ABSENT  ABSENT
[ L ][ I ][ G ][ N ][ E ] → OK      OK      OK      OK      OK
-----

Tentative 3 : lever

[ T ][ A ][ B ][ L ][ E ] → ABSENT  ABSENT  ABSENT  PRESENT  OK
[ L ][ A ][ M ][ P ][ E ] → OK      ABSENT  ABSENT  ABSENT  OK
[ L ][ E ][ V ][ E ][ R ] → OK      PRESENT  ABSENT  ABSENT  ABSENT
[ L ][ O ][ G ][ E ][ R ] → OK      ABSENT  OK      PRESENT  ABSENT
[ L ][ U ][ I ][ R ][ E ] → OK      ABSENT  PRESENT  ABSENT  OK
[ L ][ I ][ V ][ R ][ E ] → OK      OK      ABSENT  ABSENT  OK
-----

Tentative 4 : logger
```

[T][A][B][L][E] → ABSENT	ABSENT	ABSENT	PRESENT	OK
[L][A][M][P][E] → OK	ABSENT	ABSENT	ABSENT	OK
[L][E][V][E][R] → OK	PRESENT	ABSENT	ABSENT	ABSENT
[L][O][G][E][R] → OK	ABSENT	OK	PRESENT	ABSENT

Tentative 5 : luire

[T][A][B][L][E] → ABSENT	ABSENT	ABSENT	PRESENT	OK
[L][A][M][P][E] → OK	ABSENT	ABSENT	ABSENT	OK
[L][E][V][E][R] → OK	PRESENT	ABSENT	ABSENT	ABSENT
[L][O][G][E][R] → OK	ABSENT	OK	PRESENT	ABSENT
[L][U][I][R][E] → OK	ABSENT	PRESENT	ABSENT	OK

Tentative 6 : livre

[T][A][B][L][E] → ABSENT	ABSENT	ABSENT	PRESENT	OK
[L][A][M][P][E] → OK	ABSENT	ABSENT	ABSENT	OK
[L][E][V][E][R] → OK	PRESENT	ABSENT	ABSENT	ABSENT
[L][O][G][E][R] → OK	ABSENT	OK	PRESENT	ABSENT
[L][U][I][R][E] → OK	ABSENT	PRESENT	ABSENT	OK
[L][I][V][R][E] → OK	OK	ABSENT	ABSENT	OK

Vous avez utilisé tous vos essais.

Partie perdue.

Mot secret : ligne